Стек протоколов. Прикладной уровень

Как приложение проходит путь от имени до HTTP-ответа

Зачем разработчику разбираться в сети

Приложение исправно

- код запущен;
- процесс отвечает;
- тесты проходят.

Пользователь видит сбой

- имя не разрешается;
- соединение не устанавливается;
- ответ теряется или задерживается.

Пользователь наблюдает не код отдельно, а всю систему доставки запроса.

Зачем этот курс

> Выпускник курса понимает, как сеть влияет на поведение приложения, знает типичные классы сетевых отказов, умеет собрать первичные наблюдения и способен предметно взаимодействовать с SRE и сетевыми инженерами.

Только никому не говорите, но настоящая цель курса: после курса SRE и сетевику не должно хотеться сразу прибить разработчика.

Definition of Done всего курса

Прослеживает

путь взаимодействия и связывает симптом с уровнем системы

Наблюдает

собирает первичные данные и отделяет факты от предположений

Обосновывает

предсказывает влияние сети и выбирает следующий шаг

Взаимодействует

понимает границы технологий и приходит к коллегам с фактами

Рабочая инженерная модель — не квалификация сетевого инженера.

Сегодня — первый шаг к этому результату

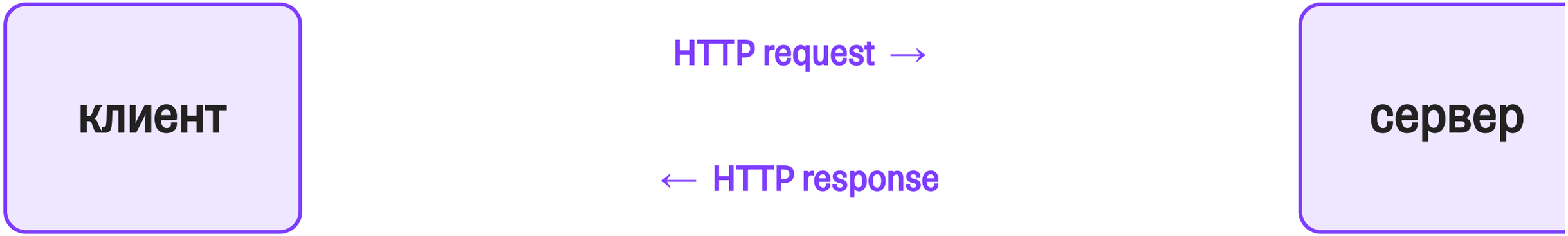
1. **Прочитать** базовый HTTP-запрос и ответ.
2. **Проследить** путь от URL через DNS и сеть до HTTP-ответа.
3. **Различить** сбой DNS, соединения, тайм-аут и HTTP-код.
4. **Выбрать** следующее наблюдение вместо угадывания причины.

Начнём с того, чем вы уже пользуетесь

`https://shop.example/catalog`

Что должно произойти после ввода этого URL?

HTTP начинается с простого обмена



Клиентом может быть браузер, мобильное приложение или другой backend.

URL задаёт цель обращения

```
https://example.com/catalog?page=2#reviews
```

SCHEME	HOST	PATH	QUERY	FRAGMENT
https	**example.com**	**/catalog**	**page=2**	**reviews**
порт 443 по умолчанию	имя узла	путь к ресурсу	параметры запроса	состояние клиента

#fragment остаётся на клиенте и обычно не попадает в HTTP-запрос.

HTTP-запрос можно прочитать

```
GET /catalog?page=2 HTTP/1.1
Host: example.com
Accept: application/json
                                                                    пустая строка
```

ЧАСТЬ СООБЩЕНИЯ	В ПРИМЕРЕ	ЧТО ОЗНАЧАЕТ
method	GET	что сделать
request target	/catalog?page=2	к какому ресурсу обратиться
version	HTTP/1.1	правила формата сообщения
field lines	Host · Accept	метаданные запроса
пустая строка	—	завершает поля; дальше может быть тело

HTTP-ответ сообщает результат

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 17
                                                                    пустая строка
{"items":[1,2,3]}
```

ЧАСТЬ СООБЩЕНИЯ	В ПРИМЕРЕ	ЧТО ОЗНАЧАЕТ
version	HTTP/1.1	формат сообщения
status code	200	класс результата
reason phrase	OK	текстовое пояснение
field lines	Content-Type · Content-Length	метаданные ответа
пустая строка	—	завершает поля
content	JSON	представление ресурса

Если видим status code, HTTP-ответ уже был сформирован.

HTTP-ошибка — тоже полученный ответ

2xx	успех	запрос обработан
3xx	перенаправление	нужно дальнейшее действие
4xx	результат запроса	сервер указывает на проблему запроса
5xx	ошибка сервера	сервер не смог выполнить запрос

ДИАГНОСТИЧЕСКАЯ ГРАНИЦА

`HTTP 503`

Ответ получен

Взаимодействие дошло до HTTP.

`нет status code`

Ответ не получен

Это другой класс результата.

Stateless не означает «состояния нет»

НТТР-ВЗАИМОДЕЙСТВИЕ

Каждый запрос можно рассматривать независимо

`request A` `request B`

СОСТОЯНИЕ ПРИЛОЖЕНИЯ

Может сохраняться между запросами

cookie / token · application storage · external service

Stateless описывает протокольное взаимодействие, а не всю архитектуру системы.

HTTP-семантика переживает смену версии

HTTP/1.1

текстовое framing

TCP

→

HTTP/2

binary framing

TCP

→

HTTP/3

binary framing

QUIC

ОБЩАЯ СЕМАНТИКА methods · status codes · fields · resources

HTTP показывает, что задаёт протокол

01

Роли

кто с кем взаимодействует

02

Сообщения

как устроен формат

03

Семантика

что означают поля и действия

04

Порядок

какая последовательность допустима

05

Условия

что происходит при ошибках

Протокол — договорённость о сообщениях, их смысле и порядке взаимодействия.

RFC описывают Интернет

RFC

Request for Comments

Нумерованные документы об интернет-стандартах, протоколах, процедурах, исследованиях и инженерном опыте.

1969

Steve Crocker выпустил RFC 1 как рабочую заметку проекта ARPANET.

КАК ЧИТАТЬ RFC РАЗРАБОТЧИКУ

Номер ≠ стандарт

В серии есть Standards Track, BCP, Informational, Experimental и Historic.

Текст не переписывают

Изменения публикуют новым RFC со связями Updates или Obsoletes.

Начинайте с metadata

Проверьте status, stream, связанные RFC и errata.

HTTP описывает не весь путь

Мы записали HTTP-запрос. Но как его байты попали на другой компьютер?

> HTTP описывает **смысл прикладного обмена**.
>
> Он не решает сам задачи адресации, доставки между сетями и физической передачи.

Уровни разделяют ответственность

application	Что означает запрос	`HTTP messa`
transport	Как доставить между процессами	`TCP stre`
internet	Как пройти между сетями	`IP pack`
link	Как пройти локальный участок	`Ethernet fra`
medium	Как передать сигнал	`bits / sign`

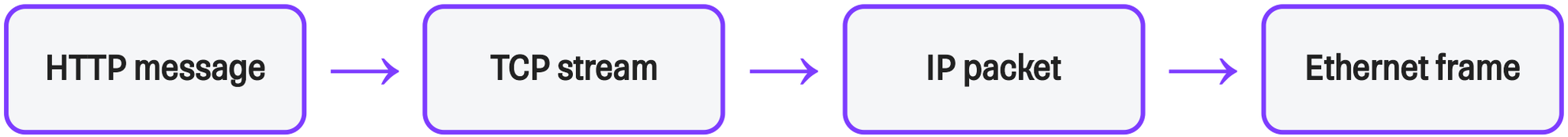
На каком уровне наблюдение перестало соответствовать ожиданию?

OSI объясняет, TCP/IP помогает работать

OSI: подробная карта		Рабочая модель курса	
L7 · application L6 · presentation L5 · session	→	application	смысл обмена
L4 · transport	→	transport	доставка между процессами
L3 · network	→	internet	доставка между сетями
L2 · data link	→	link	локальная доставка
L1 · physical	→	medium	сигнал

Это две модели описания, а не два разных сетевых стека.

Один запрос — несколько оболочек



Каждый уровень добавляет служебную информацию для своей зоны ответственности.

Устройства читают только нужные уровни

client	switch	router	server
L7	**L2**	**L3**	**L7**
создаёт HTTP-сообщение	пересылает Ethernet frame	маршрутизирует IP packet	обрабатывает HTTP-сообщение

Приложение знает имя `shop.example`.

Но IP-уровню нужен адрес назначения. Где его взять?

DNS превращает имя в адрес

`shop.example` → `192.0.2.10`

1 → N

одному имени могут соответствовать несколько адресов

адрес меняется

результат разрешения не обязан быть постоянным

имя ≠ адрес

приложению дано имя, сети нужен адрес назначен

DNS-имя читается справа налево

www.example.com.

← от корня к конкретному имени

ЧАСТЬ ИМЕНИ	В ПРИМЕРЕ	ЧТО ОЗНАЧАЕТ
host label	www	конкретное имя внутри зоны
domain	example.com	доменная зона
TLD	com	домен верхнего уровня
root	.	завершающая точка обычно скрыта

DNS хранит типизированные записи

A

IPv4 address

AAAA

IPv6 address

CNAME

canonical name alias

NS

authoritative name servers

MX

серверы, принимающие почту для домена

TXT

текстовые данные механизмов

DNS — распределённая база типизированных записей, а не только «имя → IP».

Resolver — это не один сервер

`getaddrinfo("www.example.com")` — библиотечный вызов. Приложение передаёт имя операционной системе, но не обязано само отправлять DNS-пакет.

01

application

передаёт имя

02

stub resolver

API операционной системы

03

recursive resolver

проверяет кэш и ищет ответ

Ответ есть в кэше	возвращает его без нового поиска
Ответа в кэше нет	может обратиться к `root → TLD → authoritative DNS`

Конкретный путь зависит от ОС, локальных кэшей и конфигурации resolver.

DNS-запрос и ответ можно наблюдать

QUERY

A www.yandex.ru

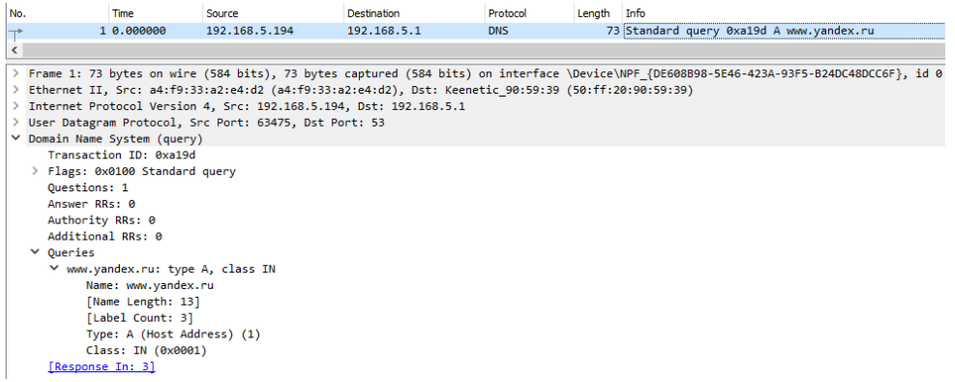
Клиент сообщает имя и тип нужной записи.

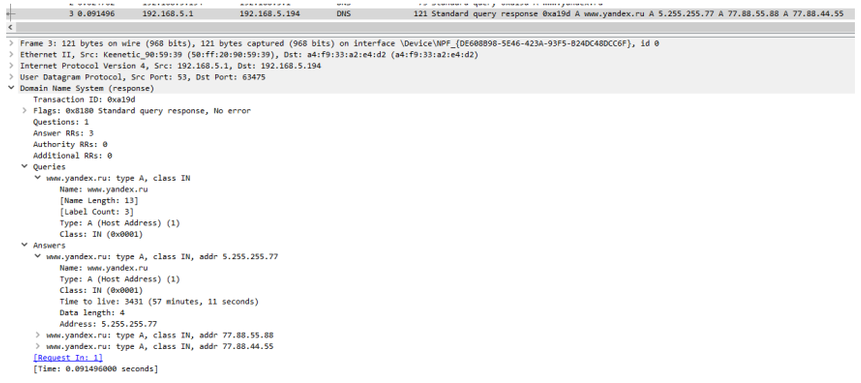
RESPONSE

A 5.255.255.77 · TTL 31

Сервер возвращает адрес и срок жизни ответа в кэше.

Ethernet IPv4 UDP DNS





TTL задаёт срок хранения DNS-ответа

TTL · Time to Live сколько секунд закэшированный DNS-ответ можно использовать без нового запроса.



t = 0

Ответ закэширован

`A 192.0.2.10`

TTL = 300 секунд

t = 120

Authoritative изменён

`A 192.0.2.20`

Кэш ещё возвращает 192.0.2.10

t > 300

TTL истёк

`A 192.0.2.20`

Следующий запрос обновляет запись

Изменение в authoritative DNS не становится видимым всем клиентам одновременно.

DNS использует не только UDP/53

UDP/53

частый путь

обычный запрос и ответ

TCP/53

штатная поддержка

не аварийный обходной путь

DoT

DNS over TLS

защищённый транспорт

DoH

DNS over HTTPS

DNS внутри HTTP

DNSSEC проверяет подлинность DNS-данных, но не заменяет транспорт.

Теперь можно собрать весь путь

01

URL

Извлечь scheme, host, port и path

02

DNS

Получить адрес назначения

03

Соединение

Установить связь с нужным процессом

04

HTTP request

Передать прикладное сообщение

05

HTTP response

Получить и прочитать результ

Каждый следующий этап возможен только после результата предыдущего.

Результат показывает достигнутый этап

01	02	03	04
DNS	**Соединение**	**HTTP**	**Содержимое**
`answer`	`established`	`response`	`expected`
`NXDOMAIN`	`refused`	`incomplete`	`unexpected`
`timeout`	`timeout`	`timeout`	

Последний подтверждённый результат сужает область поиска, но ещё не называет причину.

Текст ошибки сужает область поиска

Сначала фиксируем, что уже произошло. Причину формулируем только как следующую проверяемую гипотезу.

СООБЩЕНИЕ	ЧТО УЖЕ ДОКАЗАНО	ГДЕ ПРОДОЛЖИТЬ ПРОВЕРКУ
`Could not resolve host`	**приложение не получило адрес**	resolver, DNS-ответ и конфигурация имени
`Connection refused`	**попытка соединения получила отказ**	адрес, порт и источник отказа
`Operation timed out`	**ожидаемый результат не пришёл до дедлайна**	сначала определить, на каком этапе истёк таймер
`HTTP/1.1 503 Service Unavailable`	**HTTP endpoint или посредник сформировал ответ**	сервер, proxy и доступность upstream

Сообщение помогает выбрать следующую проверку, но само по себе не доказывает первопричину.

Гипотеза — ещё не наблюдение

Факт

HTTP-клиент не получил ответ за 5 секунд.

Что непосредственно наблюдали?

→

Интерпретация

Сработал тайм-аут клиента.

Что означает наблюдение?

→

Гипотеза

Пакеты блокирует firewall.

Что теперь нужно проверить?

Гипотеза предлагает проверку, но не заменяет наблюдение.

Диагностика начинается с фактов

01

Факт

зафиксировать наблюдение

02

Этап

найти последний достигнутый

03

Гипотезы

назвать несколько совместимых

04

Проверка

различить возможные причины

05

Действие

проверить или передать факт

Если следующий шаг вне вашей зоны — передайте коллегам факты, а не назначенного виновника.

РАСШИРЯЕМ МОДЕЛЬ

Но ведь в мире есть не только HTTP?

HTTP	получить представление ресурса	`request → response`
DNS	найти типизированные данные по имени	`query → answer`
SMTP	передать почтовое сообщение дальше	`store → forward`
IMAP	работать с почтовым ящиком на сервере	`состояние + синхронизация`
POP3	получить сохранённые сообщения	`простой доступ`
FTP	передавать файлы	`управление + данные`

Разные задачи — разные роли, сообщения и последовательности взаимодействия.

Одно письмо проходит через несколько протоколов

Клиент отправителя

создаёт сообщение

`SMTP submission` →

Сервер отправителя

по DNS MX находит следующий сервер

`SMTP relay` →

Сервер получателя

сохраняет сообщение

Клиент получателя

`IMAP` — работа с ящиком и синхронизация

`POP3` — более простое получение сообщений

Пользовательский сценарий собирается из нескольких протоколов.

Письмо доставлено. Но что находится внутри?

```
From: alice@example.com
To: bob@example.net
Subject: Отчёт
Content-Type: ???

текст
HTML
изображение
PDF-файл
```

- В какой кодировке написан текст?
- Это обычный текст или HTML?
- Где границы частей сообщения?
- Как передать бинарное вложение?

SMTP доставляет сообщение. Представление содержимого — отдельная задача.
Следующая неделя: MIME, multipart, кодировки и сериализация.

Шесть мыслей после лекции

→ HTTP описывает прикладной обмен, но не весь путь.

→ Протокол задаёт сообщения, их смысл и порядок.

→ Один пользовательский сценарий может использовать несколько протоколов.

→ Стек разделяет зоны ответственности.

→ DNS — отдельный этап со своими ролями и кешами.

→ Диагностический вывод строится из наблюдений.

На семинаре модель станет инструментом

→ Восстановим порядок этапов запроса.

→ Прочитаем HTTP- и DNS-наблюдения.

→ Отделим факт от гипотезы.

→ Выберем следующий диагностический шаг.

→ Сформулируем предметное обращение к другой команде.